

DEPARTMENT OF CSE

A.Y. 2026 – 2027

B.Tech. III Year Odd Semester

COURSE: CNAD (24CND3101)

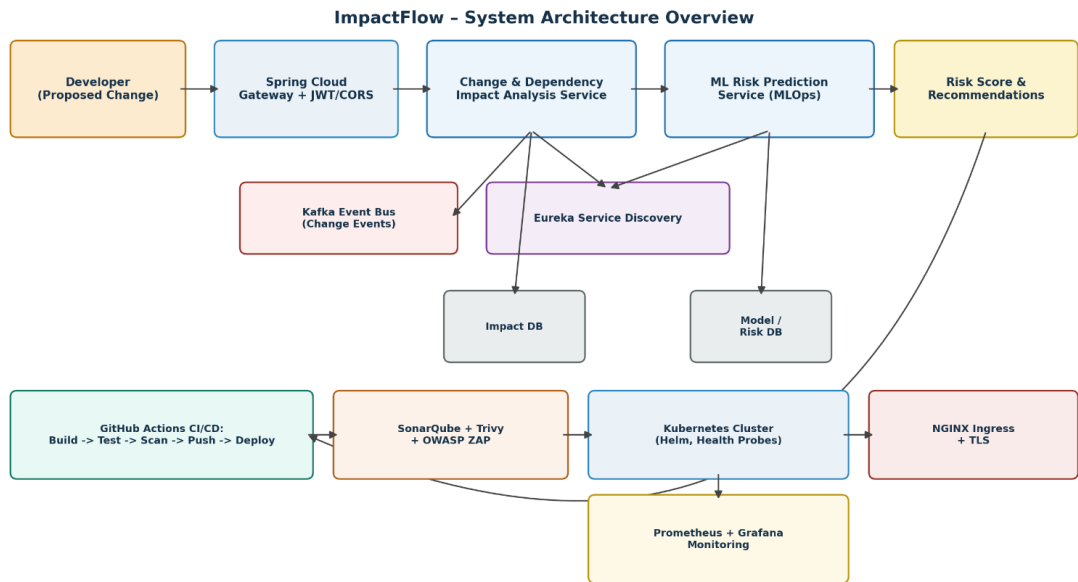
PROJECT ABSTRACT SUBMISSION

TITLE OF THE PROJECT	ImpactFlow – AI-Powered Adaptive Software Change Impact & Risk Prediction Platform	
S. No.	University ID	Name
1	2420030457	P. Sowmya
2	2420030477	N. Amrutha Lekha
Name of the Guide		Dr. P. Srilatha

Abstract

Modern software systems evolve continuously, with development teams pushing frequent changes to meet business and user demands. In large systems built using distributed and microservices-based architectures, even a small code change can affect multiple interconnected services, leading to unexpected failures, downtime, or security vulnerabilities. Developers and DevOps teams often lack visibility into which components, services, or modules a given change is likely to affect before it is deployed. Traditional CI/CD pipelines validate code correctness through testing but do not explicitly assess the impact or risk a change introduces to the broader system. This gap between change submission and its downstream consequences increases the likelihood of production incidents, rollback efforts, and delayed releases, highlighting the need for a system that can proactively assess change risk before deployment. To address this need, this project proposes ImpactFlow, a cloud-native platform that analyzes proposed software changes and predicts their potential impact and risk prior to deployment. The platform examines factors such as the changed components, their dependencies, historical change patterns, and code complexity metrics to identify services and modules likely to be affected by a given change. Based on this analysis, an AI/ML-based prediction model generates a risk score along with recommendations, enabling developers to make informed decisions before pushing changes into production. The system design draws on Advanced Software Engineering principles by structuring development around Agile/Scrum practices and by treating change impact and risk analysis as a core software engineering activity. The prediction component follows an MLOps workflow encompassing data preparation, model training, versioning, deployment, and monitoring, ensuring that the risk-prediction model can be retrained and improved as new change data becomes available. Testing and secure development practices are embedded throughout, in line with DevSecOps principles, so that security checks are treated as an integral part of the development lifecycle rather than an afterthought. The platform itself is implemented as a set of Spring Boot microservices communicating through REST APIs with well-defined service contracts, coordinated using Eureka for service discovery and Spring Cloud Gateway for routing and request handling. Where asynchronous processing is beneficial, such as propagating change events for downstream analysis, Kafka-based event-driven communication is used. Each microservice maintains its own database following the database-per-service pattern, and access to the platform is secured using JWT-based authentication with Spring Security, along with CORS configuration to control cross-origin access. Unit, integration and contract testing are used to validate individual services and their interactions. On the infrastructure side, each microservice is containerized using Docker and deployed on Kubernetes, with Helm charts used to manage deployment configurations across environments. A CI/CD pipeline built with GitHub Actions automates the build, test, security scan, image push and deployment stages, incorporating SonarQube for code quality analysis, Trivy for container vulnerability scanning, and OWASP ZAP for dynamic security testing. Kubernetes health probes and rolling deployment strategies support reliable service availability, while NGINX Ingress with TLS secures external access. Prometheus and Grafana are used to monitor service health and system performance in real time. In the proposed workflow, a developer submits a proposed change, after which ImpactFlow analyzes the change and its dependencies, identifies the relevant affected services, and applies the ML model to compute a risk score and associated recommendations. The change can then be tested and deployed through the platform's CI/CD and cloud-native infrastructure with greater confidence. Overall, ImpactFlow aims to demonstrate how software engineering, service-oriented architecture, and cloud-native DevSecOps practices can be combined to build a practical, deployable system that helps development teams anticipate the consequences of software changes before they reach production.

Project Architecture/Project Overview:



Tools and Technologies Used for the Project:

Category	Tools / Technologies
Frontend	React.js
Backend	Java Spring Boot (microservices); Python FastAPI (ML risk-prediction service)
Database	MySQL (database-per-service pattern)
Version Control	Git, GitHub
Containerization	Docker
Container Orchestration	Kubernetes / Minikube
Package Management	Helm
CI/CD	GitHub Actions
Security Scanning	Trivy (container scanning), SonarQube (SAST), OWASP ZAP (DAST)
GitOps	Not used in the core implementation — deployment is handled directly via the GitHub Actions CI/CD pipeline
Cloud Platform	Local-first (Minikube); AWS / Azure / GCP considered as an optional extension for cloud deployment
API Testing	Postman
Development IDE	Visual Studio Code / IntelliJ IDEA
Monitoring	Prometheus, Grafana
Documentation	GitHub, Markdown

Signature of the Guide

Signature of the Course
Coordinator

HOD